codigo

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Aqui está a análise passo a passo do código fornecido no arquivo `codigo.py` : PY

1. Importação de Módulos

O script importa bibliotecas nativas do Python focadas em comunicação de rede, manipulação de processos e sistema de arquivos: PY

- `socket` : Utilizado para criar conexões de rede TCP/IP.
- `subprocess` : Permite executar comandos diretamente no interpretador de comandos (shell) do sistema operacional hospedeiro.
- `json` : Usado para serializar e desserializar os dados trocados pela rede.
- `os` : Utilizado para interagir com o sistema operacional (como navegação de diretórios).
- `base64` : Utilizado para codificar e decodificar arquivos enviados ou baixados em formato binário.
- `sys` : Usado para controle de fluxo e encerramento do script.

2. Inicialização e Conexão de Rede (`__init__`)

Na inicialização da classe `Backdoor` , o script estabelece uma conexão de rede TCP (`socket.AF_INET` , `socket.SOCK_STREAM`) com um endereço IP específico (`192.168.43.38`) na porta `4444` . PY

Esse comportamento é característico de conexões do tipo **reversas** (*reverse shell*), onde o script executado na máquina vítima busca ativamente se conectar a um servidor ou ouvinte controlado pelo atacante.

3. Funções de Manipulação e Execução

O código implementa métodos que fornecem controle remoto avançado e furtivo ao operador: PY

`execute_system_commmand` : Executa qualquer comando enviado pelo invasor utilizando `subprocess.check_output(command, shell=True)` , permitindo pleno controle via linha de comando do sistema operacional (como CMD, PowerShell ou Bash).

`change_working_directory_to` : Altera o diretório de trabalho atual (`os.chdir`), permitindo que o invasor navegue livremente pelas pastas do sistema. PY

`write_file` **e** `read_file` : Permitem realizar o *upload* (salvar arquivos recebidos e decodificados de Base64) e o *download* (ler e codificar arquivos locais em Base64 para envio) entre a máquina vítima e o invasor. PY

4. Loop Principal de Controle (`run`)

O método `run` mantém um loop infinito (`while True`) aguardando comandos via rede por meio de `reliable_receive` . PY

Ele processa comandos lógicos específicos:

`exit` : Encerra a conexão e fecha o processo. PY

`cd` : Altera o diretório de trabalho. PY

`download` / `upload` : Gerencia a transferência de arquivos. PY

Outros comandos: Qualquer outra entrada é interpretada como um comando de sistema e repassada ao shell operacional por meio de `execute_system_commmand` , tendo o resultado enviado de volta ao invasor através de `reliable_send` . PY

Conclusão: Benigno ou Malware?

Este arquivo é um MALWARE.

Trata-se de um script clássico de **Backdoor / Reverse Shell** em Python. Ele foi projetado exclusivamente para fins maliciosos de acesso remoto não autorizado, permitindo que um agente externo execute comandos arbitrários no sistema, navegue por diretórios e roube ou insira arquivos na máquina onde estiver sendo executado. PY

Qual é a categoria principal deste malware?

A categoria principal deste malware é **Backdoor** (frequentemente classificado também como uma **Reverse Shell** ou um **Trojan de Acesso Remoto - RAT**).

Justificativa da Categoria:

Backdoor (Porta dos Fundos): O código foi desenhado para criar um canal de acesso não autorizado, permitindo que um operador externo assuma o controlo do sistema infetado contornando os métodos normais de autenticação.

Reverse Shell (Consola Reversa): Em vez de esperar por ligações externas (o que seria facilmente bloqueado por firewalls comuns), o script inicia ativamente uma ligação de rede a partir da máquina vítima para o IP e a porta do atacante (`192.168.43.38:4444`). PY

Funcionalidades de RAT: Inclui comandos que oferecem plena versatilidade de gestão remota, tais como:

- Execução arbitrária de comandos no sistema operativo através da shell. PY
- Navegação livre pelo sistema de ficheiros (`os.chdir`). PY
- Capacidade de roubar ou injetar ficheiros na máquina alvo através de codificação em `base64` (*upload* e *download*). PY